Lecture Notes: OpenMP — Parallel Regions

Lecture Note — HPC 1 — Università di Trieste, 2025–2026
Luca Tornatore — INAF
luca.tornatore@inaf.it

These notes accompany the slides on "Parallel Regions" and the associated example codes in the `parallel_regions/` folder of the course repository. They are meant to be read alongside the codes; I will reference them often, and you should compile, run, and modify them. Throughout, I assume you have already been introduced to the concept of OpenMP and to the fork-join execution model; if not, please revisit the introductory lecture notes first.

Disclaimer: these notes have been assembled by Claude, and then revised by the author, from the lecture transcript under the explicit prompt of sticking to what have been said in the lecture and found on the lecture's slides and example codes.

Lecture Notes: OpenMP — Parallel Regions

1. Recap: processes and threads: let's introduce OpenMP
2. The parallel region
   - 2.1 The general syntax of an OpenMP directive
3. The threads factory: where do the threads live?
   - 3.1 The virtual memory layout of a multi-threaded process
4. Thread identification
5. Controlling the number of threads
6. Conditional parallel regions: the `if` clause
7. Specialising execution within a parallel region
   - 7.1 The `single` construct
   - 7.2 The `masked` construct (and the deprecated `master`)
   - 7.3 `single` vs `masked`: when to use which
   - 7.4 The `critical` construct
   - 7.5 The `atomic` construct
   - 7.6 The `barrier` construct
8. The ordering of threads
9. Shared and private variables: the scoping rules
   - 9.1 The `private` clause — and why I advise against it
   - 9.2 Pointers: a subtlety
   - 9.3 The `default(none)` clause
   - 9.4 The wrong way: `01_simple_pr_wrong.c`
10. The data-sharing clauses in detail
    - 10.1 `firstprivate`
    - 10.2 `lastprivate`
    - 10.3 `threadprivate`
    - 10.4 `copyin`
    - 10.5 `copyprivate`
11. Nested parallelism
12. Work assignment: a preview
13. Internal Control Variables and the OpenMP environment
14. Summary and practical guidelines

Reference: example codes

1. Recap: processes and threads: let’s introduce OpenMP

Before we talk about parallel regions, let us be precise about terminology — because getting sloppy here generates confusion later.

A **process** is the bundle of a running program and all the resources the operating system has granted to it: CPU time, virtual address space (code, data, stack, heap), file descriptors, network sockets, and so on. You can have many processes running the same executable; each one is completely independent, with its own virtual memory map and its own OS-level identity (the PID).

A **thread** is an independent flow of execution *within* a process. Every process has at least one thread — thread zero, the one that starts in `main()`. Threads within the same process share the code segment and the heap, but each thread has its own stack, its own instruction pointer (IP), and its own stack pointer (SP). The fact that each thread has a separate IP is what allows them to execute different parts of the code simultaneously; the fact that they share the heap and the global data is what makes communication between threads cheap — and also what makes it dangerous, as we shall see.

OpenMP is a *shared-memory* parallel programming model. It operates within a single process: you use compiler directives (`#pragma omp ...`) and library calls to fork threads, assign them work, synchronise them, and join them back. The hardware requirement is that all threads can see the same physical memory — which means, in practice, that OpenMP lives within a single node (or within a single NUMA domain, if you are not careful about placement; we'll discuss that in the NUMA lecture).

Remind: this is fundamentally different from MPI, where each MPI rank is a separate process with its own address space, and communication requires explicit message-passing.

2. The parallel region

To use OpenMP constructs and run-time functions, you need first to include `#include <omp.h>` and to compile with `-fopenmp` (that is quite standard, but different compilers may have different openmp switches). The core construct in OpenMP is the **parallel region**. In C, it looks like:

```
#pragma omp parallel
{
    // ... code that every thread will execute ...
}
```

When execution reaches the `#pragma omp parallel` directive, the runtime *forks* a pool of threads (the "team"). Every thread in the team — including thread 0, which is the continuation of the original thread — executes the code block delimited by the braces. At the closing brace, an **implicit barrier** is present: all threads synchronise, and after the barrier only thread 0 continues into the serial region that follows.
That is called a **fork-join running model**.

A few things worth noting:

The parallel region is a code block, and in C/C++ it can be as short as a single line (in which case you don't need braces):

```
#pragma omp parallel
printf("hello world\n");
```

in practice you almost always want the braces, both for clarity and because you typically need more than one statement.

The OpenMP runtime does *not* necessarily create and destroy threads at every parallel region. For efficiency reasons, the thread pool is typically created once at the first parallel region (or at program start) and the threads are put to sleep outside of parallel regions. This is an implementation detail, but it matters when you reason about overhead: the cost of entering a parallel region is not the cost of spawning POSIX threads from scratch; it is the cost of waking the sleeping threads, setting up the team, and distributing the work. On modern systems, this is of the order of ~10 μs — small, but not zero, and definitely not negligible if you are opening parallel regions inside a tight loop.

Beside `#pragma omp parallel`, there exist other constructs that create or work within parallel regions: `#pragma omp parallel for` (which combines a parallel region with a loop work-sharing construct), `#pragma omp sections`, and `#pragma omp task`. Strictly speaking, `sections` and `task` are not themselves parallel regions — they are work-sharing or tasking constructs that operate *from inside* a parallel region. We will discuss loops in the next lecture and tasks later on.

2.1 The general syntax of an OpenMP directive

The general form in C is:

```
#pragma omp <directive> [<clause> [, <clause> [...]]]
{
    statements;
}
```

The `#pragma omp` prefix tells the compiler that what follows is to be considered an OpenMP directive. If you compile without `-fopenmp` (or the equivalent flag for your compiler), the `#pragma` is silently ignored — which is a design feature: a well-written OpenMP program should compile and run correctly as a serial program too.

In C++, since the OpenMP 5.1 standard, there is also the attribute syntax:

```
[[ omp :: directive(clause, ...) ]]
```

We will stick to the C `#pragma` syntax in these notes.

3. The threads factory: where do the threads live?

When a parallel region is opened, each thread gets its own stack: a separate region of the virtual address space, with its own base pointer (RBP) and stack pointer (RSP). You can verify this experimentally; let's look at the code in `00_scope_of_variables.c`:

```c
int i;
#pragma omp parallel
{
    int me = omp_get_thread_num();
    printf("thread %d: &i = %p, &me = %p\n", me, &i, &me);
}
```

When you run this, you will observe that `&i` is the same for all threads — because `i` was declared outside the parallel region and is therefore *shared*. But `&me` is different for every thread — because `me` was declared inside the region and lives in each thread's private stack.

The more detailed example in `00_stack_and_scope.c` digs more: it reads the RBP and RSP registers of each thread (using inline assembly) to show you the actual stack frame boundaries. You'll see that each thread's stack base is placed at a regular distance from the others, depending on the default stack reservation. The stacks are carved out of the virtual address space of the same process; they all coexist in the same page table.

You can control the per-thread stack size via the environment variable:

```bash
export OMP_STACKSIZE=8M
```

The value takes a suffix: `B`, `K`, `M`, `G` (bytes, kilobytes, megabytes, gigabytes). If no suffix is given, it's interpreted as kilobytes.

Thread count limits. The maximum number of threads is system-dependent. On Linux, it is ultimately bounded by the amount of physical memory divided by the per-thread overhead (the kernel's `task_struct`, the stack reservation, etc.).
You can check the OS limit with `cat /proc/sys/kernel/threads-max`. In practice, you will not want more threads than physical cores — having more threads than cores means context-switching and time-sharing, which is the opposite of what we are trying to achieve: parallelism.

3.1 The virtual memory layout of a multi-threaded process

If you reconstruct the output of `00_stack_and_scope.c`, you'll see something like this structure in the virtual address space:

```
[low addresses]
  code segment
  data segments (globals, BSS)
  heap (grows upward via malloc/mmap)
  ...
  thread N stack  (private variables of thread N)
  ...
  thread 2 stack  (private variables of thread 2)
  thread 1 stack  (private variables of thread 1)
  thread 0 stack  ≡ original process stack
  kernel space
[high addresses]
```

Each thread stack has a reserved region (the "max stack size"); the stacks are laid out in the virtual space separated by approximately that amount. Shared variables — those declared outside the parallel region — live either in the process stack (if they are local variables of `main` or the calling function) or in the heap (if `malloc`-ed), or in the global variables segments.
All threads access them at the same address. Private variables live in each thread's own stack, at addresses that are far apart from each other.
So, that is what “shared” and “private” mean: a **shared variable** is such because all the threads see the same memory address, while **private variables** are such because every threads uses a different memory address of that.

This picture is a slight simplification, of course — the actual layout depends on the runtime, the OS, ASLR, and so on — but it is a correct mental model for what happens at the thread level.

4. Thread identification

Inside a parallel region, each thread can query its own identity:

```
int myid    = omp_get_thread_num();   // returns 0, 1, ..., nthreads-1
int nthreads = omp_get_num_threads(); // total threads in this team
```

These are the two most basic OpenMP runtime calls.

`omp_get_num_threads()` returns the number of threads *in the innermost parallel region from which it is called*. If called from a serial region, it returns 1 — which makes sense, since there's only one thread running.

Also note that `omp_get_thread_num()` returns the thread's ID within the current team. In a nested parallel region, the IDs restart from 0 for the inner team.

The `helloworld.c` example shows the simplest possible use:

```
#pragma omp parallel
{
    int myid = omp_get_thread_num();
    printf("hello world from thread %d\n", myid);
}
```

If you run this several times, you will likely get the greetings in different orders each time. This is normal and expected: threads are independent entities, and the order of their execution is non-deterministic unless you explicitly enforce it.

5. Controlling the number of threads

By default, OpenMP spawns as many threads as there are available hardware threads (cores × SMT factor). You can override this in several ways, listed here in increasing order of precedence:

1. **Environment variable**: `export OMP_NUM_THREADS=4` — sets the default for all parallel regions.
2. **Runtime function**: `omp_set_num_threads(4);` — changes the default from that point forward in the code.
3. **Directive clause**: `#pragma omp parallel num_threads(4)` — overrides everything, but only for that specific region.

There is a subtlety with `omp_set_num_threads()`: if the `OMP_DYNAMIC` environment variable is set to `true` (or `omp_set_dynamic(1)` has been called), the runtime may use *fewer* threads than requested if it deems appropriate. If `OMP_DYNAMIC` is `false` (the default on most implementations), the number you request is the number you get.

See the examples `03a_num_of_threads.c` and `03b_num_of_threads.c` for hands-on experiments with these mechanisms.

6. Conditional parallel regions: the `if` clause

You can make the creation of a parallel region conditional:

```
#pragma omp parallel if(amount_of_work > threshold)
{
    // ... work ...
}
```

If the condition evaluates to zero (false), the parallel region is *not* created: only thread 0 executes the code block, serially. If the condition is nonzero (true), the full team is spawned as usual.

The overhead of entering a parallel region is of the order of ~10 μs. If the work inside the region would take, say, 1 μs in serial, then you are paying more for the thread management than you gain from parallelism. The `if` clause lets you fall back to serial execution when the workload is too small to justify the overhead.

This is a pragmatic and often overlooked optimisation. In real codes — especially those with adaptive algorithms where the amount of work per step varies — conditional parallel regions can make a significant difference.

Of course, that really matters only inside loops.

7. Specialising execution within a parallel region

Inside a parallel region, not everything needs to be executed by all threads. Sometimes, a piece of code is inherently serial (only one thread should do it); sometimes, you want to select a specific subset of threads. OpenMP provides several constructs for this.

7.1 The `single` construct

```
#pragma omp single
{
    // only one (arbitrary) thread executes this
}
```

The `single` construct means: one thread from the team will execute this block; which one is implementation-defined (you don't know and you shouldn't care). The other threads wait at an **implicit barrier** at the end of the block. When the barrier is crossed, a **memory flush** for all shared variables in scope is also performed — meaning that any shared variable modified inside the `single` block will be visible to all threads after the barrier (see the lecture on OpenMP memory model to understand what a `flush` actually is).

This is the correct way to do inherently serial work inside a parallel region — for instance, getting the number of threads and storing it in a shared variable:

```
int nthreads;
#pragma omp parallel
{
    #pragma omp single
    nthreads = omp_get_num_threads();

    // at this point, all threads see the correct value of nthreads,
    // thanks to the implicit barrier + flush at the end of single.
}
```

Without the `single`, *all* threads would call `omp_get_num_threads()` and write to the same shared variable — which is a data race. It happens to be benign in this particular case (they all write the same value), but it is formally undefined behaviour, and forming the habit of writing correct code is worth more than saving one line.

A more involved example appears in the slides: a producer-consumer loop where one thread reads data and the others process it:

```
#pragma omp parallel
{
    while (!done) {
        #pragma omp single
        {
            data_size = get_data_size();
            data = malloc(...);
            get_new_data(data);
        }
        // barrier here: everyone sees data

        determine_my_subset(data_size, &mydata);
        process_my_subset(data_size, mydata);

        #pragma omp barrier   // wait for all to finish processing
        #pragma omp single
        {
            free(data);
            done = check_for_more_data();
        }
    }
}
```

Note the explicit `#pragma omp barrier` before the second `single`. Without it, one thread could enter the second `single` and call `free(data)` while other threads are still processing. The barrier ensures that all threads have finished with the data before it is freed. This is a pattern you will see again and again.

7.2 The `masked` construct (and the deprecated `master`)

```
#pragma omp masked
{
    // only thread 0 executes this (by default)
}

#pragma omp masked filter(3)
{
    // only thread 3 executes this (if it exists)
}
```

The `masked` construct is the "deterministic cousin" of `single`: instead of letting the runtime choose which thread executes, you select the thread(s) explicitly via the `filter` clause. Without `filter`, it defaults to `filter(0)`, meaning thread 0.

The `filter` expression is evaluated by each thread; a thread executes the block if and only if the expression equals its thread ID:

```
// all threads execute (each sees filter == its own id):
#pragma omp masked filter(omp_get_thread_num())
{ ... }

// only even-numbered threads:
int tid = omp_get_thread_num();
#pragma omp masked filter( (tid % 2 == 0) ? tid : -1 )
{ ... }
```

There is one very important difference with `single`: **`masked` has no implicit barrier at the end**, and therefore no implicit memory flush. If you modify a shared variable inside a `masked` block, other threads are *not* guaranteed to see the updated value unless you add an explicit `#pragma omp barrier` afterwards.

When you use `single`, you get safety by default (barrier + flush). When you use `masked`, you get speed (no synchronisation overhead) but you must manage visibility yourself.

The `master` construct, which selected thread 0 exclusively, is deprecated from OpenMP 5.1 in favour of `masked`.

7.3 `single` vs `masked`: when to use which

The rule of thumb:

- If you need the result of the single-threaded work to be visible to all threads immediately → use `single` (barrier + flush included).
- If you are assigning work to a specific thread (or subset) and the other threads don't need to wait → use `masked` (cheaper, no synchronisation).
- If you just want thread 0 to print something and nobody else cares → `masked` is fine.

7.4 The `critical` construct

```
#pragma omp critical
{
    // only one thread at a time executes this
}
```

The `critical` construct is a mutual exclusion mechanism: all threads will eventually execute the block, but only one at a time. The others queue up and wait their turn. At the entry point there is a lock (with a memory flush of all shared variables in scope); there is *no* synchronisation at the exit — threads leave as soon as they finish, without waiting for the others.

This is a serialisation point: every thread must pass through it one after another, in unspecified mutual order. Note that using a critical region is a severe serialization of your workflow.

Named critical regions. By default, OpenMP has a single global critical section. All unnamed critical regions in the entire program are considered part of the same lock. This means that if you have two unrelated critical regions:

```
#pragma omp critical
{ /* region A */ }

#pragma omp critical
{ /* region B */ }
```

then a thread inside region A will block another thread trying to enter region B, even though they protect different data.

To avoid this, use named critical regions:

```
#pragma omp critical(update_counter)
{ counter++; }

#pragma omp critical(log_message)
{ fprintf(logfile, ...); }
```

Named regions have independent locks: different names can be entered simultaneously by different threads.

7.5 The `atomic` construct

```
#pragma omp atomic update
sum += myid;
```

The `atomic` construct protects a *single memory location of a basic type* using hardware-level atomic instructions (like `lock add` on x86). It is much cheaper than a critical region — there is no lock acquisition, no thread queuing, just a cache-line lock for the duration of the instruction.

However, `atomic` is limited to specific forms of assignment: simple read, write, update, or capture operations on scalar variables. You cannot put an arbitrary code block inside `atomic`, exactly because it is meant to protect the access to a specified memory location, and not the execution of a code block.

The clauses are: `read`, `write`, `update` (the default), and `capture`. The `update` form covers all the common cases: `x++`, `x--`, `x binop= expr`. The `capture` form additionally stores the old or new value.

The `data_race.c` example demonstrates the problem:

```
int sum = 0;
#pragma omp parallel
{
    int myid = omp_get_thread_num();
    sum += myid;  // DATA RACE!
}
```

The `sum += myid` involves several operations at the machine level: load `sum` from memory into a register, add `myid`, store the result back. If two threads interleave these operations, one thread's update can be lost. The fix is either `#pragma omp atomic update` before the line, or `#pragma omp critical` around it.

An important subtlety: **`atomic` and `critical` protect different things and are not mutually exclusive.** `critical` protects a *code region*; `atomic` protects a *memory location*. If one thread is inside a critical region updating variable `x`, another thread executing `#pragma omp atomic` on the same `x` is *not* blocked by the critical region. They are independent mechanisms. If you need both kinds of protection, you must use the same mechanism consistently.

7.6 The `barrier` construct

```
#pragma omp barrier
```

A barrier is the most rude synchronisation tool: all threads stop and wait until every thread in the team has arrived. When a thread leaves the barrier, it is guaranteed that all other threads have at least reached it. A memory flush of all shared variables in scope happens at the barrier.

Use barriers the least possible: they are expensive (all threads idle while waiting for the slowest), and you loose completely your parallelism at that point.
Note that the barriers are a potential source of deadlocks for poorly designed workflows: if not every thread can reach them (for instance, if a barrier is inside an `if` branch that only some threads take).

The implicit barrier at the end of a parallel region, and at the end of a `single` block, are barriers. You can suppress the implicit barrier with the `nowait` clause (`#pragma omp single nowait`), but do this only if you are certain that no thread needs to see the results before continuing.

8. The ordering of threads

Let’s look at an exercise with pedagogical purposes: how to force threads to print their greetings in order (thread 0 first, then thread 1, etc.). This is not something you'd do in production code — it defeats the purpose of parallelism — but it forces you to reason about synchronisation.

First attempt (wrong): `04_order_of_threads_wrong.c`

```
int order = 0;
#pragma omp parallel
{
    int myid = omp_get_thread_num();
    #pragma omp critical
    if (order == myid) {
        printf("greetings from thread %d\n", myid);
        order++;
    }
}
```

This doesn't work because each thread enters the critical section only once. If thread 2 happens to enter before thread 1, it finds `order == 0` (or 1) ≠ 2, fails the test, and leaves — permanently. It never gets another chance.

Second attempt (correct but ugly): `05a_order_of_threads.c`

```
int order = 0;
#pragma omp parallel
{
    int myid = omp_get_thread_num();
    int done = 0;
    while (!done) {
        #pragma omp critical
        if (order == myid) {
            printf("greetings from thread %d\n", myid);
            order++;
            done = 1;
        }
    }
}
```

Now the `while` loop gives each thread repeated chances to check whether it's its turn. This works, but it's a busy-wait: all threads are spinning on the critical section, which is inefficient and wastes cycles.

**Third attempt (using `masked`): `05b_order_of_threads.c`

```
int order = 0;
#pragma omp parallel
{
    int nthreads = omp_get_num_threads();
    int myid = omp_get_thread_num();
    while (order < nthreads) {
       #pragma omp masked filter(order)
        {
          printf("greetings from thread %d\n", myid);
          order++;
        }
       #pragma omp barrier
    }
}
```

Fourth attempt (using `ordered`): `05c_order_of_threads.c`

```
int nthreads;
#pragma omp parallel
{
    #pragma omp single
    nthreads = omp_get_num_threads();

    #pragma omp for ordered
    for (int i = 0; i < nthreads; i++)
        #pragma omp ordered
        printf("greetings from thread %d\n", omp_get_thread_num());
}
```

The `ordered` clause on a `for` loop (we'll see the `for` construct n the next lecture) forces the iterations to execute in the order of the loop index. This is the clean solution — when it's applicable. We will discuss `#pragma omp for` in detail in the next lecture; for now, just note that `ordered` exists and that it imposes a strict sequential ordering on loop iterations, which has obvious performance implications.

Note also the use of `single` (not `masked`) for getting `nthreads`: since there is an implicit barrier at the end of `single`, all threads will see the correct value of `nthreads` before entering the `for` loop. If we had used `masked` without a barrier, some threads might read `nthreads` before thread 0 had written it — and proceed into the loop with a stale value (possibly the initialisation value 1, meaning only one iteration would be scheduled).

9. Shared and private variables: the scoping rules

This is where most OpenMP bugs come from. Understanding what is shared and what is private is absolutely fundamental.

The default rule is simple:

- Variables declared **outside** the parallel region → **shared** (all threads see the same address).
- Variables declared **inside** the parallel region → **private** (each thread has its own copy, living in its own stack).
- Global variables → **shared** (with the exception of `threadprivate`, discussed below).
- Dynamically allocated memory → the **pointer** follows the rules above; the **pointed-to memory** on the heap is inherently accessible to all threads (shared).
- Loop indices on `#pragma omp for` → **private** (automatically).

Let me stress again about what "shared" and "private" mean here:

- A variable is **shared** if its address (`&var`) is the same for all threads in the team.
- A variable is **private** if its address is different for each thread.

9.1 The `private` clause — and why I advise against it

You can explicitly declare variables as private:

```
int i, j;
#pragma omp parallel private(i)
{
    // i is private: each thread has its own copy.
    // But its value is UNINITIALISED — not copied from the serial region.
    // j is shared.
}
```

This creates a new variable in each thread's stack with the same *name* as the original `i`, but a different address and an undefined initial value. The original `i` in the serial region is untouched and inaccessible from within the parallel region.

My personal advice: **avoid the `private` clause.** In my view, it is a relic from Fortran77-era thinking, and it is a source of confusion. If you need a per-thread variable, just declare it inside the parallel region:

```
int i;
// ... using i in serial code ...

#pragma omp parallel
{
    int j;  // private by declaration, clear and obvious
    // ... using j ...
    // i is still accessible here, and it's shared.
}
```

This is cleaner: `j` is manifestly a new variable, local to the parallel region. There is no ambiguity about what `j` refers to. With `private(i)`, you create a situation where the name `i` refers to two different memory locations depending on whether you're inside or outside the region — which is exactly the kind of cognitive overhead that leads to bugs in real code.

Of course, there are cases where the private-like clauses (`firstprivate`, `lastprivate`) are genuinely useful; we'll get to those.

9.2 Pointers: a subtlety

Consider:

```
double *array = malloc(N * sizeof(double));

#pragma omp parallel
{
    // array is shared: all threads see the same pointer value,
    // which points to the same heap memory.
    array[omp_get_thread_num()] = 42.0;  // fine, distinct elements
}
```

versus:

```
double *array = malloc(N * sizeof(double));

#pragma omp parallel private(array)
{
    // array is private: each thread has its own pointer variable.
    // But it's UNINITIALISED — it does NOT point to the malloc-ed memory!
    // Dereferencing it is undefined behaviour.
    array[0] = 42.0;  // CRASH (or corruption)
}
```

When you declare a pointer as `private`, only the pointer object itself (8 bytes on x86-64) is privatised. The pointed-to memory is *not* duplicated. The private copy is uninitialised — it does not inherit the value of the original pointer. So the new private `array` points to garbage.

If you really want a private pointer named as the shared pointer that still points to the original data, use `firstprivate`:

```
#pragma omp parallel firstprivate(array)
{
    // array is private, but initialised to the value of the original pointer.
    // So it still points to the malloc-ed memory.
    array[omp_get_thread_num()] = 42.0;  // fine
}
```

9.3 The `default(none)` clause

It is considered good practice by many people to use:

```
#pragma omp parallel default(none) private(j) shared(array, N)
```

This forces the compiler to reject any variable in the scope that is not explicitly listed as `shared` or `private`.

9.4 The wrong way: `01_simple_pr_wrong.c`

Let's have a look at this code, which contains a couple of obvious bugs:

```
int nthreads;
int my_thread_id;

#pragma omp parallel
{
    my_thread_id = omp_get_thread_num();  // DATA RACE
    nthreads     = omp_get_num_threads(); // USELESS RACE
    printf("greetings from thread %d\n", my_thread_id);
}
```

Both `nthreads` and `my_thread_id` are shared (declared outside the region). All threads write to `my_thread_id` in an unspecified order — so by the time any thread prints it, the value is whatever the last writer happened to store, which depends on scheduling. The result is that you may get the same thread ID printed multiple times and some IDs not printed at all.

For `nthreads`, every thread writes the same value, so the race is "benign" in the sense that the final value is always correct — but it is still formally a data race, which is undefined behaviour in the C memory model.

The correct version (`01_simple_parallel_region.c`) declares `my_thread_id` inside the region (making it private) and uses `#pragma omp masked` for `nthreads`:

```
int nthreads = 1;
#pragma omp parallel
{
    int my_thread_id = omp_get_thread_num();  // private, correct
   #pragma omp masked
    nthreads = omp_get_num_threads();         // only thread 0 writes

   #pragma omp barrier  // ensure nthreads is visible to all
    printf("greetings from thread %d out of %d\n", my_thread_id, nthreads);
}
```

Note the barrier: without it, some threads might read `nthreads` before thread 0 has written it (since `masked` has no implicit barrier). With the barrier, all threads are guaranteed to see the updated value.

10. The data-sharing clauses in detail

10.1 `firstprivate`

```
int i = 42;
int *array = malloc(...);

#pragma omp parallel firstprivate(i, array)
{
    // i is private and initialised to 42
    // array is private and initialised to point to the same heap memory

    array[omp_get_thread_num()] = i + omp_get_thread_num();
    array = NULL;  // safe: only destroys this thread's local copy of the pointer
}
// array in the serial region is still valid
```

`firstprivate` means: create a private copy for each thread, but initialise it with the value the variable has at the point of entering the parallel region. For scalars, the value is copied. For pointers, the pointer value (i.e., the address it points to) is copied — not the pointed-to data.

See `09_clauses__firstprivate.c` for a working example.

10.2 `lastprivate`

```
double last_result;

#pragma omp parallel
{
    #pragma omp for lastprivate(last_result)
    for (int j = 0; j < N; j++)
        last_result = compute(j);
}

// last_result == compute(N-1), the value from the last iteration
```

`lastprivate` applies to `for`, `sections`, and `taskloop` constructs. Each thread has a private copy (uninitialised, like `private`), but after the construct, the value from the *last iteration* (or last section) is copied back to the original variable in the serial scope.

"Last iteration" means the iteration with the highest index — which is well-defined even when the iterations are distributed across threads in an arbitrary order.

See `09_clauses__lastprivate.c`.

10.3 `threadprivate`

```
int myN;
double *array;
#pragma omp threadprivate(myN, array)

#pragma omp parallel
{
    array = malloc(...);
    myN = omp_get_thread_num();
}
// serial region: array and myN still exist for each thread

#pragma omp parallel
{
    // array still points to the previously allocated memory
    // myN still has the previously assigned value
}
```

`threadprivate` is special: it applies to **global** (or file-scope static) variables and makes them private *across the entire lifetime of the program*, not just within a single parallel region. Each thread gets its own persistent copy that survives between parallel regions.

This is useful when you have library code that uses global state (random number generators, for instance) and you need each thread to have its own copy of that state.

Restriction: when using `threadprivate`, the number of threads must remain constant across all parallel regions (dynamic thread creation is not allowed).

See `09_clauses__threadprivate.c`.

10.4 `copyin`

```
int golden_values[3];
#pragma omp threadprivate(golden_values)

for (int i = 0; i < N; i++) {
    get_golden_values();  // fills thread 0's copy
    #pragma omp parallel copyin(golden_values)
    {
        // each thread's golden_values is now a copy of thread 0's
    }
}
```

`copyin` is the broadcast mechanism for `threadprivate` variables: at the start of the parallel region, the value from thread 0 (the "master") is copied into every thread's private copy.

See `09_clauses__copyin.c` and `09_clauses__copyin__clarify.c`.

10.5 `copyprivate`

```
#pragma omp parallel
{
    double seed[2];

    #pragma omp single copyprivate(seed)
    {
        // one thread initialises seed
        seed[0] = compute_seed_0();
        seed[1] = compute_seed_1();
    }
    // at this point, ALL threads have a copy of seed
    // (propagated from the thread that executed the single block)
}
```

`copyprivate` applies only to `single`. After the single block is executed, the private variables listed in `copyprivate` are copied from the executing thread to all other threads. The copy happens before any thread leaves the implicit barrier at the end of `single`.

This is different from `copyin` in that it works with *local* private variables (not only threadprivate), and it operates *within* a parallel region rather than at its entry.

See `09_clauses__copyprivate.c`.

11. Nested parallelism

OpenMP permits nesting of parallel regions:

```
#pragma omp parallel num_threads(4)
{
    // level 0: 4 threads
    #pragma omp parallel num_threads(2)
    {
        // level 1: each of the 4 threads spawns 2 more → up to 8 threads
    }
}
```

Whether nested parallelism is actually *active* depends on the environment variables:

- `OMP_MAX_ACTIVE_LEVELS=n` — the maximum depth of active nesting (default is usually 1, meaning inner regions are serialised).
- `OMP_NUM_THREADS=N0,N1,...` — comma-separated list giving the team size at each nesting level.
- The older `OMP_NESTED=true|false` is deprecated since OpenMP 5.0 but still supported by most compilers.

In practice, nested parallelism is rarely useful. The typical use case is when you have outer parallelism over coarse-grained tasks and inner parallelism within each task. But the overhead of creating nested teams is non-trivial, and you risk oversubscribing the hardware (e.g., 4 outer threads × 4 inner threads = 16 threads on an 8-core machine → contention).

See `02_stack_nested_parallelism.c` for an example that shows how the variable scoping works across nesting levels: `k` (declared in the serial region) is shared at level 0 and visible (still shared) at level 1; `me` (declared at level 0) is private at level 0 and, from the perspective of each level-1 team, it behaves as a shared inherited variable; `j` (declared at level 1) is private to each level-1 thread.

12. Work assignment: a preview

So far, we have seen how to open parallel regions and control which threads execute what. But we have not yet distributed *different work* to *different threads*. In the examples above, all threads execute the same code (except for the branching based on thread ID).

The most common way to distribute work is through **domain decomposition** — all threads do the same operations, but on different portions of the data. The workhorse for this is `#pragma omp for`, which we will cover in the next lecture. The other approach is **functional decomposition** — different threads do different things. You can achieve this manually by branching on the thread ID:

```
#pragma omp parallel
{
    int myid = omp_get_thread_num();
    if (myid % 3 == 0)
        result = heavy_work_0();
    else if (myid % 3 == 1)
        result = heavy_work_1();
    else
        result = heavy_work_2();
}
```

Or you can use the `sections` construct (which we won't cover in this lecture), or the `task` construct (which we will cover in the advanced course). The manual approach is flexible but error-prone; the structured constructs are usually preferable.

13. Internal Control Variables and the OpenMP environment

OpenMP behaviour is governed by **Internal Control Variables** (ICVs) — conceptual variables that you access via environment variables (before the program starts) or via runtime functions (during execution). "Conceptual" means you don't care where they're stored; you only care how to get and set them.

ICVs have different scopes:

- **Global** scope: applies to the whole program, set once at startup. (Example: `omp_cancellation`.)
- **Device** scope: applies per-device, relevant when targeting GPUs or co-processors. (Examples: `omp_stacksize`, `omp_wait_policy`, `omp_schedule`.)
- **Task** scope: local to each task (and a parallel region is, at the task level, a task). When new tasks are created, they inherit the parent's ICVs. Changes within a task do not propagate backwards. (Examples: `omp_num_threads`, `omp_max_active_levels`, `omp_dynamic`, `omp_proc_bind`.)

The most useful ones for day-to-day work:

Environment variable	ICV	What it controls
`OMP_NUM_THREADS`	nthreads	Default team size
`OMP_STACKSIZE`	stacksize	Thread stack reservation
`OMP_SCHEDULE`	run-schedule	Default loop schedule (static, dynamic, etc.)
`OMP_PROC_BIND`	binding	Thread-to-core affinity policy
`OMP_PLACES`	placement	Where threads can run (cores, sockets, etc.)
`OMP_MAX_ACTIVE_LEVELS`	max-active-levels	Maximum nesting depth
`OMP_DYNAMIC`	dynamic	Whether runtime can adjust team size
`OMP_WAIT_POLICY`	waiting-policy	What idle threads do (ACTIVE = spin, PASSIVE = sleep)

We'll encounter several of these again in the lectures on loops and NUMA awareness.

Some useful runtime functions, at a glance:

- `omp_get_num_threads()` — number of threads in the current team
- `omp_get_thread_num()` — ID of the calling thread
- `omp_set_num_threads(n)` — set default team size from this point on
- `omp_get_max_threads()` — upper bound for the next team size
- `omp_get_wtime()` — wall-clock time in seconds (not synchronised across threads; use barriers before timing if needed)

14. Summary and practical guidelines

Let me collect the things that, in my experience, matter most in practice:

Declare private variables inside the parallel region. Don't use `private(x)` to privatise a variable that was declared outside — it's confusing and error-prone. Just declare a new variable inside. The exception is `firstprivate`, which has a genuinely useful semantic (initialise the private copy).

Always think about the barrier. Every time you write to a shared variable, ask yourself: when will the other threads see this? If you used `single`, there's an implicit barrier. If you used `masked`, there isn't — and you probably need an explicit one.

Prefer `atomic` over `critical` when possible. `atomic` uses hardware support and is orders of magnitude cheaper than `critical`. But it's limited to simple update operations; for anything more complex, you need `critical` (or a rethinking of the algorithm).

Use named critical regions. The global unnamed critical section is a performance trap. If you have two independent critical sections, name them so they don't block each other.

Don't fight the non-determinism. The order of thread execution is not guaranteed, and trying to force it (as in our ordering exercise) is expensive and fragile. Design your algorithms so that correctness does not depend on execution order.

In the next lecture, we will move to loop parallelism — `#pragma omp for` and its scheduling clauses — which is where OpenMP truly starts to earn its keep in scientific computing.

Reference: example codes

File	Topic
`helloworld.c`	Minimal parallel region
`00_scope_of_variables.c`	Shared vs. private addresses
`00_stack_and_scope.c`	Thread stack layout, RBP/RSP probing
`01_simple_pr_wrong.c`	Data race on shared `my_thread_id`
`01_simple_parallel_region.c`	Correct use of `masked` + barrier
`02_stack_nested_parallelism.c`	Nested regions, variable visibility
`03a_num_of_threads.c`, `03b_num_of_threads.c`	Controlling team size
`04_order_of_threads_wrong.c`	Failed ordering attempt
`05a_order_of_threads.c`, `05a_order_of_threads_bis.c`	Busy-wait ordering
`05b_order_of_threads.c`, `05c_order_of_threads.c`	Ordering via `ordered`
`data_race.c`	Data race on accumulator

File	Topic
`09_clauses__firstprivate.c`	`firstprivate` usage
`09_clauses__lastprivate.c`	`lastprivate` usage
`09_clauses__threadprivate.c`	`threadprivate` globals
`09_clauses__copyin.c`, `09_clauses__copyin__clarify.c`	`copyin` broadcast
`09_clauses__copyprivate.c`	`copyprivate` with `single`